Sensores e Sinais

Aula Prática: Transformada de Z e Filtros Digitais

Paulo Pedreiras, Pedro Fonseca, Luis Silva, DETI/UA/IT

2025-2026

Informações da Aula

- **Duração:** 2 horas em sala + 2 horas de trabalho extra-aula
- **Tópico:** Transformada de Z e Filtros Digitais
- **Baseado em:** Oppenheim et al, Capítulo 10, e Smith et al, Capítulos 14 e 33
- **Preparação prévia:** Aula TP relativa à Transformada de Z

Objetivos de Aprendizagem

- Compreender a Transformada de Z
- Saber calcular a transformada de Z e a ROC de alguns sinais elementares
- Saber analisar a resposta de sistemas usando a Transformada de Z, incluindo filtros digitais
- Saber usar ferramentas computacionais (Octave/Matlab) no contexto de análise e síntese de sistemas com Transformadas de Z

---

1 Parte 1: Exercícios em aula (1h45m)

1.1 Exercício 1.1: Cálculo Básico da Transformada Z

Cálculo Manual:

Calcule a Transformada Z e a Região de Convergência (ROC) para as seguintes sequências:

a) $x[n] = (0.5)^n u[n]$

b) $x[n] = -(0.8)^n u[-n-1]$

c) $x[n] = \delta[n] + 2\delta[n-1] + \delta[n-2]$

d) $x[n] = n(0.99)^n u[n]$ (aplicar propriedade, slide TP 11, exp. 10.5.8)

Verificação em Octave:

O Octave possui um package denominado "symbolic"que é uma ferramenta extremamente útil para vários contextos, incluindo o estudo e aplicação da Transformada Z.

Este package permite realizar cálculos algébricos exatos, em contraste com os cálculos numéricos tradicionais, o que é particularmente valioso quando se trabalha com expressões matemáticas complexas.

No caso concreto da Transformada Z, o package simbólico facilita várias tarefas importantes:

- Permite calcular transformadas Z de forma analítica, manipulando expressões como variáveis simbólicas em vez de valores numéricos, o que ajuda a compreender melhor as relações matemáticas subjacentes;
- Torna mais simples a expansão em frações parciais, uma técnica fundamental para calcular transformadas Z inversas, pois o package pode fatorizar polinómios e decompor funções racionais automaticamente (ver função "[r, p, k] = residue(b, a)");
- Permite verificar cálculos manuais de forma rápida. Os estudantes podem resolver problemas analiticamente no papel e depois confirmar os resultados usando o Octave, o que reforça a aprendizagem e reduz erros;
- A capacidade de simplificar expressões algébricas complexas também é crucial quando se trabalha com funções de transferência que envolvem múltiplos polos e zeros.

O script seguinte ilustra o uso deste package. Como trabalho para casa devem completar o script para verificar os cálculos das restantes alíneas deste exercício.

Nota: o package "symbolic" pode não estar instalado por defeito. Se for esse o caso, digite o comando **pkg install -forge symbolic** na janela de comandos do Octave.
O package "symbolic" recorre ao python e necessita do package python "sympy". Se o script assinala este erro deve abrir um terminal no Linux e digitar o comando **pip install sympy**.

```
% Exercicio 1.1 - Verificacao usando computacao numerica e simbolica
pkg load symbolic
clear

% Ex 1_1 a): x[n] = (0.5)^n u[n]
% Verificacao numerica
n = 0:20;
x_a = (0.5).^n;
figure(1);
stem(n, x_a);
title('x[n] = (0.5)^n u[n]');
xlabel('n'); ylabel('x[n]');
grid on;

% Verificacao simbolica
syms n;
f=(0.5)^n;
F=ztrans(f);
display('Transformada de Z, Ex.1.1 a)');
display(F)

```

```
% Ex 1_3 a): x[n] = delta[n] + 2*delta[n-1] + delta[n-2]
% Verificacao numerica
% ----
% ----

% Verificacao simbolica
syms n;
f=kroneckerDelta(n,0)+2*kroneckerDelta(n,1)+kroneckerDelta(n,2);
F=ztrans(f);
display('Transformada de Z, Ex.1.1 c)');
display(F)
```

Listing 1: Exercício 1.1 - Verificação

1.2 Exercício 1.2: Transformada Z Inversa

Cálculo Manual:

Determine a transformada Z inversa usando expansão em frações parciais:

a) $X(z) = \frac{2z}{(z-0.5)(z-0.3)}$, $|z| > 0{,}5$

b) $X(z) = \frac{z^2}{z^2-0.5z-0.5}$, $|z| > 1$

> Nota: quando a expressão da transformada de Z está expressa com potências positivas de z, em geral é melhor efectuar a expansão em fracções parciais sobre a expressão dividida por z, e voltar a multiplicar por z no fim. Isso permite obter directamente expressões na forma $\frac{z}{z-a}$, directamente invertíveis, sem cálculos adicionais (normalmente "time-shifts").

Implementação em Octave:

```
% Exercicio 1.2 - Transformada Z Inversa
% Parte (a)
disp("a)")
num = [2];  % O numerador correcto e [2 0]. Neste caso fez-se divisao
    por z para simplificar a inversa
den = conv([1 -0.5], [1 -0.3]);
[r, p, k] = residue(num, den);

disp('Coeficientes da Fracao Parcial (dividida por z):');
disp('Residuos:'); disp(r);
disp('Polos:'); disp(p);
disp('Termo direto:'); disp(k);
disp('Nao esquecer de multiplicar por z a fraccao final!')

% Reconstruir a sequencia
n = 0:20;
x_n = r(1)*(p(1).^n) + r(2)*(p(2).^n);
figure(121);
stem(n, x_n);
title('Transformada Z Inversa de X(z) Ex 1.2 a)');
xlabel('n'); ylabel('x[n]');
grid on;
```

Listing 2: Exercício 1.2 - Transformada Z Inversa

1.3 Exercício 1.3: Análise de Sistemas Usando a Transformada Z

Enunciado do Problema:

Um sistema LTI em tempo discreto é descrito pela equação diferença:

$$y[n] - 0.8y[n-1] = x[n] + 0.5x[n-1] \tag{1}$$

a) Determine a função de transferência $H(z)$

b) Determine os polos e zeros

c) Trace o diagrama de polos e zeros

d) Calcule as primeiras 5 amostras da resposta impulsional ($h[n], n = 0...4$), quer partindo da equação diferença inicial quer da transformada inversa de H(z). Compare ambos os resultados.

e) O sistema é estável?

Implementação em Octave:

```
% Exercicio 1.3 - Analise de Sistemas
% Coeficientes da funcao de transferencia
b = [_____];     % Numerador (zeros)
a = [_____];    % Denominador (polos)

% (b) Encontrar polos e zeros
zeros_H = roots(b);
poles_H = roots(a);

disp('Zeros:'); disp(zeros_H);
disp('Polos:'); disp(poles_H);

% (c) Diagrama de Polos e Zeros
figure(3);
zplane(b, a);
title('Diagrama de Polos e Zeros');
grid on;

% Adicionar circulo unitario para verificacao de estabilidade
hold on;
theta = linspace(0, 2*pi, 100);
plot(cos(theta), sin(theta), 'r--', 'LineWidth', 1.5);
hold off;

% (d) Resposta ao Impulso
N = 40;
impulse = [1 zeros(1, N-1)];
h = filter(b, a, impulse);

figure(4);
stem(0:N-1, h);
title('Resposta ao Impulso h[n]');
xlabel('n'); ylabel('h[n]');
grid on;

% (e) Verificacao de estabilidade
if all(abs(poles_H) < 1)
```

```
    disp('Sistema e ESTAVEL (todos os polos dentro do circulo unitario)');
else
    disp('Sistema e INSTAVEL');
end

% Resposta em Frequencia
[H, w] = freqz(b, a, 512);
figure(5);
subplot(2,1,1);
plot(w/pi, 20*log10(abs(H)));
title('Resposta em Magnitude');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Magnitude (dB)');
grid on;

subplot(2,1,2);
plot(w/pi, angle(H)*180/pi);
title('Resposta em Fase');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Fase (graus)');
grid on;
```

Listing 3: Exercício 1.3 - Análise de Sistemas

1.4 Exercício 1.4: Projeto de Filtro Digital Simples

Enunciado do Problema:

Projete e implemente um filtro de média móvel simples com 5 coeficientes:

$$y[n] = \frac{x[n] + x[n-1] + x[n-2] + x[n-3] + x[n-4]}{5} \tag{2}$$

a) Determine a função de transferência $H(z)$

b) Determine os polos e zeros

c) Analise a resposta em frequência

d) Teste o filtro com um "sinal ruidoso" (*).
Use uma sinusoide com amplitude unitária como sinal, e adicione ruido com uma variância de 0.25, que é "elevado" e bem visível.

e) Repita os passos c) e d) com usando um filtro de 3 coeficientes e de 10 coeficientes. Compare os resultados.

As alíneas a) e b) devem ser feitas manualmente. As restantes alíneas devem ser feitas em Octave.

(*) Muitos sensores reais apresentam na sua saída sinais que estão afectados por ruído e interferência. Em muitos casos (especialmente processos físicos) o ruído pode modelar-se com um grau razoável de aproximação por um sinal de média nula e com uma distribuição gaussiana (ou normal). A função do Octave "randn(M, N)" gera uma matriz com M*N amostras e variância unitária, podendo ser usada para modelar a presença de ruído, somando-se algebricamente ao sinal "puro".

Se se pretender um sinal com variância não unitária, deve multiplicar-se o vector gerado pelo randn() (com variância unitária) pela raiz quadrada da variância desejada.

Implementação em Octave:

```
% Exercicio 1.4 - Filtro de Media Movel

% (a) Funcao de transferencia
N_taps = 5;
b = ones(1, N_taps) / N_taps;
a = 1;

disp('Coeficientes da Funcao de Transferencia:');
disp('Numerador b:'); disp(b);
disp('Denominador a:'); disp(a);

% (b) Polos e Zeros
zeros_filter = roots(b);
disp('Zeros do Filtro:'); disp(zeros_filter);
disp('Polos do Filtro: z = 0 (na origem)');

figure(6);
zplane(b, a);
title('Filtro de Media Movel - Diagrama de Polos e Zeros');

% (c) Resposta em Frequencia
[H, w] = freqz(b, a, 512);
figure(7);
subplot(2,1,1);
plot(w/pi, abs(H));
title('Filtro de Media Movel - Resposta em Magnitude');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Magnitude');
grid on;

subplot(2,1,2);
plot(w/pi, angle(H));
title('Resposta em Fase');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Fase (rad)');
grid on;
```

Listing 4: Exercício 1.4 - Filtro de Média Móvel

```
% (d) Teste com sinal ruidoso
n = 0:199;
fs = 1000; % Frequencia de amostragem (Hz)
f_signal = 50; % Frequencia do sinal (Hz)
clean_signal = sin(_______);
noise = _________;
noisy_signal = clean_signal + noise;

% Aplicar filtro
filtered_signal = filter(b, a, noisy_signal);

figure(8);
subplot(3,1,1);
plot(n, clean_signal);
title('Sinal Original Limpo');
xlabel('Amostra n'); ylabel('Amplitude');
grid on;

subplot(3,1,2);
plot(n, noisy_signal);
title('Sinal Ruidoso');
xlabel('Amostra n'); ylabel('Amplitude');
grid on;

subplot(3,1,3);
plot(n, filtered_signal);
title('Sinal Filtrado');
xlabel('Amostra n'); ylabel('Amplitude');
grid on;
```

Listing 5: Exercício 1.4 - Teste com Sinal Ruidoso (continuação)

2 PARTE 2: Exercícios complementares, para resolver fora da aula

2.1 Exercício 2.1: Propriedades da Transformada Z

Cálculo Manual:

Dado $x[n] = (0{,}6)^n u[n]$ com transformada Z $X(z) = \frac{z}{z-0{,}6}$, $|z| > 0{,}6$
Use as propriedades da transformada Z para determinar:

a) $\mathcal{Z}\{x[n-2]\}$ (Deslocamento no tempo)

b) $\mathcal{Z}\{n \cdot x[n]\}$ (Multiplicação por n)

c) $\mathcal{Z}\{(0{,}5)^n x[n]\}$ (Multiplicação exponencial)

d) $\mathcal{Z}\{x[n] * x[n]\}$ onde $*$ denota convolução

Verificação em Octave:

```
% Exercicio 2.1 - Verificacao
n = 0:30;
x_n = (0.6).^n;

% (a) Deslocamento no tempo: x[n-2]
x_shifted = [0 0 x_n(1:end-2)];

% (b) Multiplicacao por n
x_mult_n = n .* x_n;

% (c) Multiplicacao exponencial
x_exp = (0.5).^n .* x_n;

% (d) Convolucao
x_conv = conv(x_n, x_n);

figure(9);
subplot(2,2,1);
stem(n, x_shifted);
title('x[n-2]');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(2,2,2);
stem(n, x_mult_n);
title('n \cdot x[n]');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(2,2,3);
stem(n, x_exp);
title('(0.5)^n \cdot x[n]');
xlabel('n'); ylabel('Amplitude');
grid on;

subplot(2,2,4);
stem(0:length(x_conv)-1, x_conv);
title('x[n] * x[n]');
```

```
xlabel('n'); ylabel('Amplitude');
grid on;
```

Listing 6: Exercício 2.1 - Verificação

2.2 Exercício 2.2: Análise de Sistema de Segunda Ordem

Enunciado do Problema:

Um filtro digital de segunda ordem tem a função de transferência:

$$H(z) = \frac{0{,}05z^2}{z^2 - 1{,}5z + 0{,}9} \tag{3}$$

a) Determine a equação diferença

b) Calcule os polos e zeros

c) Verifique a estabilidade

d) Trace a resposta ao degrau

e) Trace a resposta em frequência e identifique o tipo de filtro (passa-baixo, passa-alto, passa-banda, rejeita-banda)

Cálculo Manual:

Converta $H(z)$ para a forma de equação às diferenças através de multiplicação cruzada.

Implementação em Octave:

```
% Exercicio 2.2 - Sistema de Segunda Ordem

% Funcao de transferencia
num = [0.05 0 0];
den = [1 -1.5 0.9];

% (a) Mostrar equacao as diferencas
disp('Equacao as Diferencas:');
disp('y[n] - 1.5*y[n-1] + 0.9*y[n-2] = 0.05*x[n]');

% (b) Polos e zeros
z_zeros = roots(num);
z_poles = roots(den);

disp('Zeros:'); disp(z_zeros);
disp('Polos:'); disp(z_poles);

% (c) Estabilidade
figure(10);
zplane(num, den);
title('Filtro de Segunda Ordem - Diagrama de Polos e Zeros');

pole_magnitudes = abs(z_poles);
disp('Magnitudes dos Polos:'); disp(pole_magnitudes);

if all(pole_magnitudes < 1)
    disp('Sistema e ESTAVEL');
else
    disp('Sistema e INSTAVEL');
end

% (d) Resposta ao Degrau
N = 50;
```

```
step_input = ones(1, N);
step_output = filter(num, den, step_input);

figure(11);
stem(0:N-1, step_output);
title('Resposta ao Degrau');
xlabel('n'); ylabel('y[n]');
grid on;
```

Listing 7: Exercício 2.2 - Sistema de Segunda Ordem

```
% (e) Resposta em Frequencia
[H, w] = freqz(num, den, 512);

figure(12);
subplot(2,1,1);
plot(w/pi, 20*log10(abs(H)));
title('Resposta em Frequencia - Magnitude');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Magnitude (dB)');
grid on;

subplot(2,1,2);
plot(w/pi, unwrap(angle(H))*180/pi);
title('Resposta em Frequencia - Fase');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Fase (graus)');
grid on;

% Identificar tipo de filtro
[max_val, max_idx] = max(abs(H));
max_freq = w(max_idx);

if max_freq < pi/4
    disp('Tipo de Filtro: PASSA-BAIXO');
elseif max_freq > 3*pi/4
    disp('Tipo de Filtro: PASSA-ALTO');
else
    disp('Tipo de Filtro: PASSA-BANDA');
end
```

Listing 8: Exercício 2.2 - Resposta em Frequência (continuação)

2.3 Exercício 2.3: Projeto de Filtro Notch Simples

Enunciado do Problema:

Projete um filtro notch para remover uma interferência de 60 Hz de um sinal amostrado a 500 Hz.

Um filtro notch pode ser projetado usando a função de transferência:

$$H(z) = \frac{z^2 - 2\cos(\omega_0)z + 1}{z^2 - 2r \cdot \cos(\omega_0)z + r^2} \tag{4}$$

onde ω_0 é a frequência normalizada do notch e r controla a largura do notch (tipicamente $r = 0{,}9$–$0{,}99$).

a) Calcule ω_0 para 60 Hz com $f_s = 500$ Hz

b) Projete o filtro com $r = 0{,}95$

c) Determine os coeficientes da função de transferência

d) Trace a resposta em frequência

e) Teste com um sinal composto contendo interferência de 60 Hz

Cálculo Manual:

Calcule $\omega_0 = 2\pi f_0/f_s$ e expanda os polinómios da função de transferência.

Implementação em Octave:

```
% Exercicio 2.3 - Projeto de Filtro Notch

% (a) Calcular frequencia normalizada
fs = 500;            % Frequencia de amostragem (Hz)
f_notch = 60;        % Frequencia do notch (Hz)
omega_0 = 2*pi*f_notch/fs;

disp(['Frequencia normalizada do notch: ', num2str(omega_0), ' rad/amostra']);
disp(['Frequencia normalizada (x pi): ', num2str(omega_0/pi)]);

% (b) & (c) Projetar filtro
r = 0.95;            % Parametro de largura do notch

% Coeficientes do numerador (zeros)
b = [1 -2*cos(omega_0) 1];

% Coeficientes do denominador (polos)
a = [1 -2*r*cos(omega_0) r^2];

disp('Coeficientes da Funcao de Transferencia:');
disp('Numerador (b):'); disp(b);
disp('Denominador (a):'); disp(a);

% Polos e zeros
z_zeros = roots(b);
z_poles = roots(a);

disp('Zeros do Filtro:'); disp(z_zeros);
disp('Polos do Filtro:'); disp(z_poles);

% Diagrama de polos e zeros
figure(13);
zplane(b, a);
title('Filtro Notch - Diagrama de Polos e Zeros');
hold on;
theta = linspace(0, 2*pi, 100);
plot(cos(theta), sin(theta), 'r--', 'LineWidth', 1.5);
hold off;
grid on;

% (d) Resposta em Frequencia
[H, w] = freqz(b, a, 512);

figure(14);
subplot(2,1,1);
plot(w/pi, 20*log10(abs(H)));
title('Filtro Notch - Resposta em Magnitude');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Magnitude (dB)');
grid on;
hold on;
% Marcar frequencia do notch
plot(omega_0/pi, 20*log10(abs(freqz(b, a, omega_0))), 'ro', 'MarkerSize', 10);
legend('Resposta', 'Frequencia do Notch');
```

```
hold off;

subplot(2,1,2);
plot(w/pi, unwrap(angle(H))*180/pi);
title('Filtro Notch - Resposta em Fase');
xlabel('Frequencia Normalizada (\times\pi rad/amostra)');
ylabel('Fase (graus)');
grid on;
```

Listing 9: Exercício 2.3 - Projeto de Filtro Notch

```
% (e) Teste com sinal composto
n = 0:999;          % 1000 amostras
t = n/fs;           % Vetor de tempo

% Sinal desejado: 20 Hz
f_desired = 20;
signal_desired = sin(2*pi*f_desired*t);

% Interferencia: 60 Hz
interference = 0.8*sin(2*pi*f_notch*t);

% Sinal composto (desejado + interferencia)
signal_composite = signal_desired + interference;

% Aplicar filtro notch
signal_filtered = filter(b, a, signal_composite);

% Visualizacao
figure(15);
subplot(4,1,1);
plot(t(1:200), signal_desired(1:200));
title('Sinal Desejado (20 Hz)');
xlabel('Tempo (s)'); ylabel('Amplitude');
grid on;

subplot(4,1,2);
plot(t(1:200), interference(1:200));
title('Interferencia (60 Hz)');
xlabel('Tempo (s)'); ylabel('Amplitude');
grid on;

subplot(4,1,3);
plot(t(1:200), signal_composite(1:200));
title('Sinal Composto (Desejado + Interferencia)');
xlabel('Tempo (s)'); ylabel('Amplitude');
grid on;

subplot(4,1,4);
plot(t(1:200), signal_filtered(1:200));
title('Sinal Filtrado (apos Filtro Notch)');
xlabel('Tempo (s)'); ylabel('Amplitude');
grid on;

% Analise espectral
figure(16);
N_fft = 1024;
f_axis = (0:N_fft-1)*fs/N_fft;

% Espectro do sinal composto
X_composite = abs(fft(signal_composite, N_fft));
subplot(2,1,1);
plot(f_axis(1:N_fft/2), 20*log10(X_composite(1:N_fft/2)));
title('Espectro do Sinal Composto');
xlabel('Frequencia (Hz)'); ylabel('Magnitude (dB)');
grid on;
hold on;
plot([f_notch f_notch], ylim, 'r--', 'LineWidth', 2);
```

```
legend('Espectro', 'Frequencia de Interferencia (60 Hz)');
hold off;

% Espectro do sinal filtrado
X_filtered = abs(fft(signal_filtered, N_fft));
subplot(2,1,2);
plot(f_axis(1:N_fft/2), 20*log10(X_filtered(1:N_fft/2)));
title('Espectro do Sinal Filtrado');
xlabel('Frequencia (Hz)'); ylabel('Magnitude (dB)');
grid on;
hold on;
plot([f_notch f_notch], ylim, 'r--', 'LineWidth', 2);
legend('Espectro', 'Frequencia Removida (60 Hz)');
hold off;

% Calcular atenuacao na frequencia de interferencia
H_at_notch = freqz(b, a, omega_0);
attenuation_dB = 20*log10(abs(H_at_notch));
disp(['Atenuacao em ', num2str(f_notch), ' Hz: ', ...
      num2str(attenuation_dB), ' dB']);
```

Listing 10: Exercício 2.3 - Teste com Sinal Composto (continuação)